CHAPTER 02: The term vocabulary and postings lists

https://drive.google.com/file/d/1RcTifS8kTU2V-oO09YEs1k1tSV2FIwPo/view
https://drive.google.com/file/d/1RES3FsGfNlrNCQ9oYVnNyY0Y0mj5Pkf1/view
https://drive.google.com/file/d/1TA8QGwOCot4KHyO5X0I966w9uzpdurB8/view
GPT LINK: **https://chatgpt.com/share/697f41f4-1284-800c-96ea-967a6c33f0a0**
https://chatgpt.com/share/6999097c-3c20-800c-80bf-0feba9d098ce

⭐ Definitions

1) Word

- A **word** is a **delimited string of characters** as it appears in the text, usually separated by spaces or punctuation.
- It is the **raw form** from the document
- No processing is applied

Example (text):

```
Cats are running
```

Words:

- Cats
- are
- running

2) Term

- A **term** is a **normalized form of a word**.
- Normalization may include:
  - Lowercasing
  - Stemming or lemmatization
  - Removing punctuation
  - Handling spelling variations

- Terms represent an **equivalence class of words**.

  Example:

  - Words: `Cats`, `cats`, `CAT`
  - Term: `cat`

  📌 Terms answer: **"What concept does this word represent?"**

3) Type

- A **type** is the **unique vocabulary item** in a document or collection.
- **Class of all tokens containing the same character sequence**
  - Usually the **same as a term**
  - Represents a **class of identical tokens**

  Example (text):

  ```
  cat cat dog
  ```

  - Tokens = 3
  - Types = 2 (`cat`, `dog`)

  📌 Types answer: **"How many distinct terms are there?"**

4) Token

- A **token** is an **instance of a word (or term)** occurring in a document.
- **Each appearance** counts as a **separate token**
- Same word appearing multiple times → multiple tokens

  Example (text):

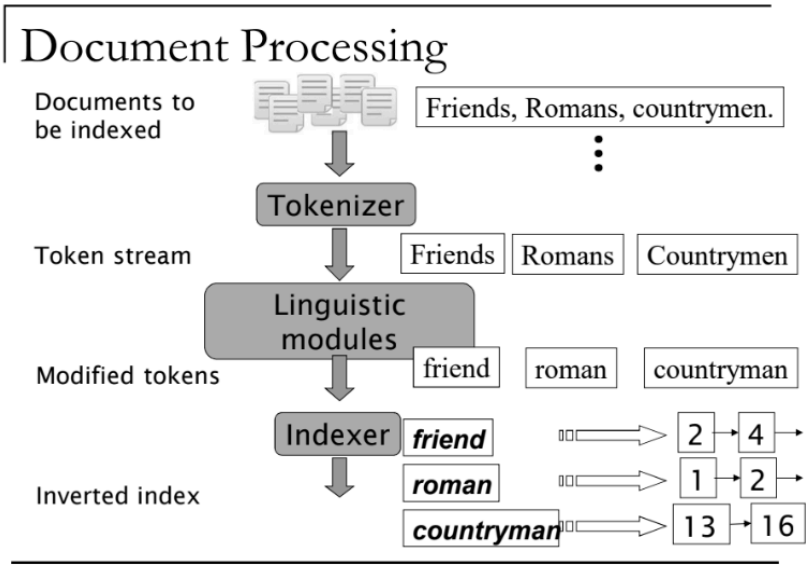
  ```
  cat cat dog
  ```

  Tokens:

  - cat (token 1)
  - cat (token 2)
  - dog (token 3)

📌 Tokens answer: **“How many times does it occur?”**

⭐ Document processing



- Docs to be indexed are **RAW TEXT DOCS** in the collection
- **Tokenizer** breaks text into **tokens** (words) → removes punctuations n splits text by spaces n delimiters
- **Linguistic modules** **normalize tokens** to produce **terms** [lowercasing, stemming, lemmatization,Stopword removal]
- **indexer** builds the **inverted index**.
- **inverted index** maps each **term** → **list of documents** in which it appears.

⭐ Challenges in Document Processing

[all problems r a type of **classification problem !!!!**]

- What **format** is it in? pdf/word/excel/html?
- What **language** is it in?
- What **character set** is in use? (CP1252, UTF-8, ...)

→ size of doc can be small(a file/email/blog) OR large (group of files)

→ a doc is a stream of strings/characters/codes ⇒ language identification is challenging!
→ **classifier** can identify language just from **100 bytes**

🪙 tokenization:

- **Tokenization** is the process through which documents are **parsed into smaller units** for processing.
- Throwing away certain chars : **punctuation!**
- process of deciding **when to emit a token** while reading a document.
- Determines where a token **starts and ends!**
- The text is split into **sequences of characters**
- These sequences are usually separated by: spaces, punctuation, special symbols
- 📌 Tokenization is a **crucial preprocessing step** in document processing.

Input: Friends, Romans, Countrymen, lend me your ears;
Output: | Friends | Romans | Countrymen | lend | me | your | ears |

Token:

→ an **instance of a sequence of characters produced** during tokenization
→ **Each occurrence is counted separately**
→ The same word appearing multiple times produces multiple tokens

issues in tokenization:

- tokenization is **not always straightforward** bec real-world text is messy
- Certain words, symbols, and formats create **ambiguity** about where tokens should start and end.
- Issues of tokenization are **language specific!** → thus requires language of the doc to be known!

1) **finland's capital , Issue**: Apostrophe (')

   Possible tokenizations:

- Finland + capital
- Finland's + capital
- Finland + 's + capital

📌 **Why it's a problem:**
Apostrophes can indicate possession or contraction. The tokenizer must decide whether to:

- remove `'s`
- keep it as part of the word
- split it into a separate token

2) **Hewlett-Packard , Issue:** Hyphenated words
Possible tokenizations:

- Hewlett + Packard
- Hewlett-Packard (single token)

📌 **Why it's a problem:**
Hyphens may:

- join a proper noun
- join two meaningful words
- Different tokenizers may treat hyphens differently, affecting search results.

4. San Francisco: one token or two?

Issue: Multi-word expressions / Named entities

Possible tokenizations:

- San + Francisco (two tokens)
- San Francisco (one token)

📌 **Why it's a problem:**
"San Francisco" is a **single entity**, but tokenization based only on spaces splits it into two, losing semantic meaning.

5. Numbers

Numbers appear in **many formats**, making tokenization difficult.

a. 3/20/91

- Could represent a **date**
- Slash `/` creates ambiguity

b. Mar. 12, 1991

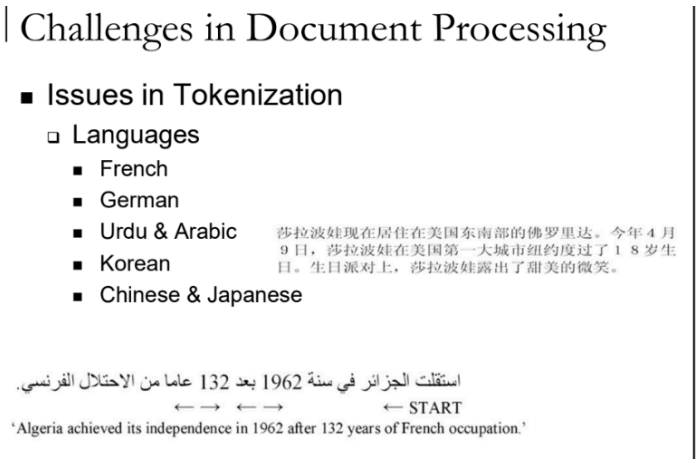
- Contains abbreviation, punctuation, and numbers
- Needs normalization to a standard date format

c. 55 B.C.

- Historical date
- Periods and uppercase letters complicate token boundaries

d. B-52

- Alphanumeric with hyphen
- Could be:
  - a model name
  - a code
  - a single concept



Tokenization becomes challenging because **different languages follow different writing systems and rules**. A single tokenization strategy does not work for all languages.

⭐Stop words:

- **Stop words** are **very common words** in a language that are **often excluded from** the **dictionary** (index).
- Intuition: they have little semantic meaning [a,an,the,to,be]
- There are a lot of them: ~30% of postings for top 30 words
- Removing them reduces index size and speeds up processing

- Using stop lists reduces number of postings
- Doesnt help removing stop words when we're doing **phrase and proximity searches**
- good compression techniques greatly reduce the cost of storing the postings for common words.

Current Trend: Moving Away from Stop Word Removal

Modern IR systems often **do not remove stop words.**

Reasons:

1. **Efficient compression techniques**
   - Advanced compression methods greatly reduce storage cost
   - Including stop words takes **very little extra space**
2. **Efficient query optimization**
   - Modern query processing techniques reduce the cost of handling stop words
   - Including stop words adds **little overhead at query time**
3. **Phrase and proximity queries**
   - Stop words are important in queries like:
     - "to be or not to be"
     - "state of the art"
   - Removing stop words can harm retrieval quality

⭐ Normalization to Terms

- **Normalization** is the process of **converting words in documents and queries into the same standard form** so that matching becomes consistent.
- Ensure that different surface forms of a word are treated as **the same term**
- Needed cuz we may write the **same word in different ways**, but they usually mean the **same thing**.
- Eg U.S.A and USA ⇒ without normalization will be treated as different words
- Result of normalization is a "**term**". It is an **entry in the IR system dictionary.**
- Multiple words can map to **one term**

Defining Equivalence Classes (Common Normalization Rules)

- deleting periods to form a term
  - ***U.S.A., USA***
- deleting hyphens to form a term
  - ***anti-discriminatory, antidiscriminatory antidiscriminatory***

⭐Normalization in Other Languages

- Normalization is especially important in **languages with accents and special characters**.
- Normalize tokens to remove diacritics

1. Accents/diacritics (French)

- Words may appear **with or without accents**.
- **Example:**
  - `résumé`
  - `resume`

📌 Users often do **not type accents**, so normalization removes them.

2. Umlauts (German)

- German words may be written in multiple forms.
- **Example:**
  - `Tübingen`
  - `Tuebingen`

3. Cedilla / Diacritics

- Special marks on characters (ç, ñ, etc.) create variations.
- 📌 These are often **removed during normalization**.

- It is often best to normalize to a **de-accented form**

Example:

- `Tuebingen`
- `Tübingen`
- `Tubingen`

➡️ All normalized to:

```
nginx

tubingen
```

Most Important Criterion in Normalization

How do users usually type their queries?

- Even if a language officially uses accents or special characters, users often **omit them while typing**
- IR systems should normalize text in a way that matches **user behavior**

⭐ Case folding

- converting **all letters to lowercase** during document and query processing.

Exceptions / Ambiguities in Case Folding

Some words **change meaning** based on capitalization.

Examples:

- **General Motors**
  - Proper noun (company name)
- **Fed vs. fed**
  - `Fed` → Federal Reserve
  - `fed` → past tense of feed
- **SAIL vs. sail**
  - `SAIL` → an organization or acronym
  - `sail` → verb or noun

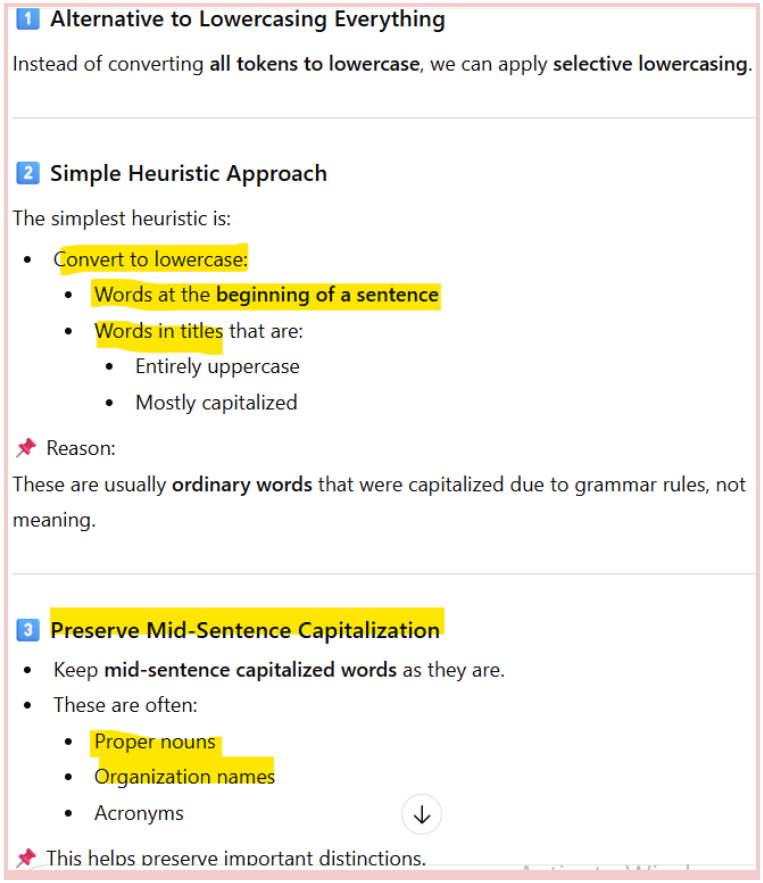
📌 **Challenge:**

Lowercasing everything may cause **loss of semantic distinction** in such cases.

⇒ despite these exceptions, It is **often best to lowercase everything**

⇒ **why?** Most users do not capitalize correctly

› Selective Case Folding in English:



1️⃣ **Alternative to Lowercasing Everything**

Instead of converting **all tokens to lowercase**, we can apply **selective lowercasing**.

2️⃣ **Simple Heuristic Approach**

The simplest heuristic is:

- Convert to lowercase:
  - Words at the **beginning of a sentence**
  - Words in titles that are:
    - Entirely uppercase
    - Mostly capitalized

📌 Reason:

These are usually **ordinary words** that were capitalized due to grammar rules, not meaning.

3️⃣ **Preserve Mid-Sentence Capitalization**

- Keep **mid-sentence capitalized words** as they are.
- These are often:
  - Proper nouns
  - Organization names
  - Acronyms

📌 This helps preserve important distinctions.

› Truecasing (More Advanced Approach)

- **machine learning sequence model.**
- The model uses multiple contextual features to decide:
  - When to lowercase
  - When to preserve capitalization

📌 Truecasing **predicts** the correct **capitalization based on context.**

⭐ Normalization to Terms: Asymmetric Expansion

What is Asymmetric Expansion?

Asymmetric expansion is an **alternative to equivalence class normalization**.

- Instead of forcing all word variants into **one single term**
- The system **expands queries or indexing differently depending on the input**

📌 It is called *asymmetric* because the **expansion depends on how the word is entered.**

Why Asymmetric Expansion is Useful

Some words have:

- singular vs plural forms
- common word vs proper noun meanings

Treating all of them as identical can lose meaning.

Example: "window / windows / Windows"

User Input	Search Terms Used
Enter: `window`	Search: `window`, `windows`
Enter: `windows`	Search: `windows`, `window`
Enter: `Windows`	Search: `Windows` only

📌 Explanation:

- `window` → generic object
- `windows` → plural form
- `Windows` → proper noun (operating system)

Asymmetric expansion preserves **semantic intent.**

Advantages & Disadvantages

- ✅ More powerful and flexible
- ❌ Less efficient (more terms searched)
- ❌ Increases query processing cost

1️⃣ What is Equivalence Classing?

In equivalence classing:

- Different words are treated as **exactly the same term**
- They are merged into **one dictionary entry**
- They share **one postings list**

Example:

```
Code

car = automobile
```

Both words map to a single term:

```
Code

car-automobile → postings list
```

📌 This is symmetric and identical.

⭐Thesauri and soundex

- **Thesaurus**: structured list of words and their semantic relationships (e.g., synonyms, related terms, broader/narrower terms).

A **thesaurus** is used to handle: **synonyms** and sometimes **variant spellings**

- Example synonyms:
  - car ↔ automobile
  - color ↔ colour

> **two main ways** to use a thesaurus in an IR system.

1) Index-time Equivalence Classes:

- synonym handling is done **while building the index**, not during search.
- **Documents** are **rewritten or expanded at indexing time**
- **Synonymous** words are **grouped into** a **single equivalence class**

How it works

- If a document contains the word `automobile`
- The system indexes it under a combined term such as:

```
car-automobile
```

- Similarly, if a document contains `car`, it is also indexed under `car-automobile`

📌 Result:

- Both `car` and `automobile` point to the **same posting list**

Advantages

- Fast query processing
- No need to expand queries at search time
- Consistent synonym handling

Disadvantages

- Index becomes larger
- Hard to modify synonym rules later
- Synonym relationships are fixed at indexing time

2) Query Expansion:

- synonym handling is done **at search time**, not during indexing.
- The index remains unchanged
- The **query is rewritten or expanded**

How it works

- If the user enters the query:

```
automobile
```

- The system automatically expands it to:

```
automobile OR car
```

- Both terms are searched in the index

📌 Result:

- Documents containing either word are retrieved

Advantages

- Flexible and easy to update
- Thesaurus can be modified without rebuilding index
- More control over query behavior

Disadvantages

- Slower query processing
- Larger intermediate result sets
- Query evaluation becomes more complex

› What is Soundex?

Soundex is a **phonetic algorithm** used to handle **spelling mistakes.**

- Groups words based on **how they sound,** not how they are spelled
- Words that sound similar are treated as belonging to the same equivalence class.

Why Soundex is Needed

Users often:

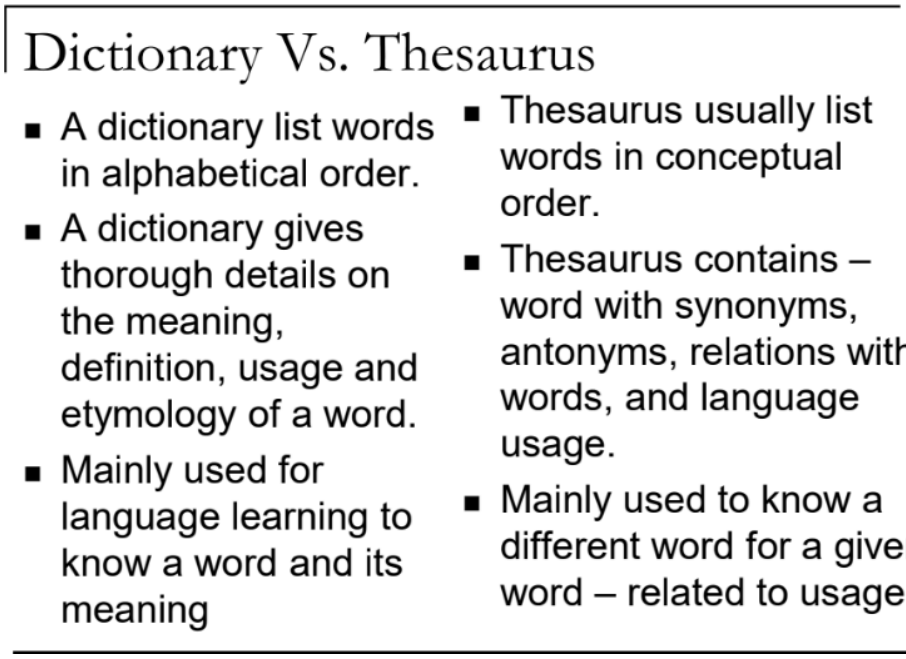
- misspell names
- spell phonetically

Example:

- Smith
- Smyth
- Smithe

➡️ All may **map to the same Soundex code**

⭐ dictionary vs thesaurus



⭐ stemming

- rule based process to transform **different form of words to a common root** or stem word
- basic way to **address dimensionality curse** in NLP application.
  > without stemming, each word form is treated as a **separate feature,** increasing vocab size, storage cost n computational complexity
- Various algos: **porter's stemmer, lovins stemmer, krovetz stemmer, RegExp**
- These algos were made by diff linguistics, so they're v subjective
- The stem may **not always be a valid dictionary word**
- Stemming **improves recall**
- refers to a crude **heuristic process** that **chops off the ends of words** in the hope of achieving this goal correctly most of the time, and often includes the **removal of derivational affixes**.
- Most commonly collapses **derivationally related words.**
- It reduces words to a common stem by **stripping** **prefixes**/**suffixes**

⭐ lemmatization

- **Lemmatization** is the process of reducing words to their **dictionary headword (lemma).**
- It performs a **proper linguistic reduction**
- The output is always a **valid dictionary word**
- It uses **vocabulary + grammar (part of speech)**
- Lemmatization reduces**: Inflectional forms + Variant grammatical forms**
- Lemmatization **preserves semantic correctness.**
- Uses **vocabulary** and **morphological analysis**.
- Usually collapses only **inflectional forms**.

📌 Inflectional forms:

- Do NOT change part of speech
- Only express grammatical variation (tense, number, etc.)

- E.g.,
  - *am, are, is → be*
  - *car, cars, car's, cars' → car*
- *the boy's cars are different colors → the boy car be different color*

- For example, in English, the verb 'to walk' may appear as 'walk', 'walked', 'walks', 'walking'. The base form, 'walk', that one might look up in a dictionary, is called the *lemma* for the word.

Q. when to use stemming or lemmatization ???????

> **stemming** when speed n efficiency is more important than accuracy, when we're working with very **large text collection**s and **slight loss of meanin**g is **acceptable**. goal is to **reduce vocabulary size quickly**

> **lemmatization** when accuracy n meaning is important!! output must be **linguistically correct ,** Context and **part of speech** matter

Stemming vs. Lemmatization

- **Stemming**
  - It is a heuristic- rule based approach, generally fast and use a single term.
  - It generates unreadable tokens.
  - Stemming algorithms err on the side of being too aggressive, sacrificing precision in order to increase recall.
- **Lemmatization**
  - It is a rigor process that uses a dictionary and uses context to determine the lemma, considered a slow approach.
  - It generates readable lexeme from the dictionary.
  - Lemmatization offers better precision than stemming, but at the expense of recall.

⭐ Porter Stemmer:

- **rule-based stemming algorithm** used to reduce words to their **stem** by systematically removing prefixes and suffixes.
- resulting stem **may not be a valid dictionary word**
- **An incoming word goes through the following stages:**
  1. **Initialization (clean-up)**
  2. **Prefix trimming phase**
  3. **Five suffix-trimming steps (Step 1 to Step 5)**

📌 **The algorithm applies rules sequentially, not all at once.**

› Initialization

- Clean the word before stemming.
- Convert the word to **lowercase**
- Keep **only letters and digits**
- Remove punctuation and special characters
- F-16 → f16

› Prefix Removal Phase

- Certain **common prefixes** are removed if present.

- Eg prefixes that r removed: kilo, micro, milli, intra, ultra, mega, nano, pico, pseudo
- kilobyte → byte
- megabyte → byte

› Porter Step 1 (Plural & Basic Suffix Handling)

- Remove **"es"** from words that end in "**sses**" or **"ies"**
  [passes --› pass, cries --› cri**]**

- Remove **"s"** if the **next-to-last letter is not "s"**
  [runs --› run, fuss --› fuss**]**

- Remove **"ed" if** the word **has a vowel** and **ends** with **"eed"**
  [agreed --› agre, freed --› freed**]**

- Replace **trailing "y" with "i"** if the **word has a vowel**
  [fly → fly (unchanged) , satisfy → satisfi **]**

› Porter Step 2 (Suffix Replacement – Longer Suffixes)

One suffix is replaced with another, shorter suffix

Examples:

Suffix	Replaced With	Example
tional	tion	conditional → condition
ization	ize	nationalization → nationalize
iveness	ive	effectiveness → effective
fulness	ful	usefulness → useful
ousness	ous	nervousness → nervous
entli	ent	fervently → fervent
biliti	ble	sensibility → sensible

› Porter Step 3 (Further Suffix Simplification) ›

With what is left, replace any suffix on the left with suffix on the right

Examples:

Suffix	Result	Example
icate → ic	fabricate → fabric	
ative → —	combative → comb	
alize → al	nationalize → national	
ical → ic	tropical → tropic	
ful → —	faithful → faith	
ness → —	harness → har	

Porter Step 4 (Remove Remaining Standard Suffixes)

Removes many common suffixes if conditions are met.

⇒ al, ance, ence, er, ic, able, ible, ant, ement, ment, ent, sion, tion, ou, ism, ate, iti, ous, ive, ize, ise

› Porter Step 5 (Final Cleanup)

Remove **trailing "e"** if the word does **not end in a vowel**

hinge --› hing

free --› free

Experimental Results of Porter's Stemmer

Given:

- Vocabulary size: **10,000 words**

Reductions:

- Step 1: 3597 words
- Step 2: 766 words
- Step 3: 327 words
- Step 4: 2424 words
- Step 5: 1373 words
- Not reduced: 3650 words

Final result:

- Final vocabulary size: **6370 stems**
- Vocabulary reduced by **about one-third**

📌 This shows how effective Porter's Stemmer is in reducing dimensionality.

Advantages of Porter's Stemmer

- Fast and efficient
- Reduces vocabulary size significantly
- Improves recall in IR systems
- Easy to implement

Disadvantages

- Stems may not be real words
- Can over-stem or under-stem
- Does not consider word meaning or context

Sample text: Such an analysis can reveal features that are not easily visible from the variations in the individual genes and can lead to a picture of expression that is more biologically transparent and accessible to interpretation

Porter stemmer: such an analysi can reveal featur that ar not easili visibl from the variat in the individu gene and can lead to a pictur of express that is more biolog transpar and access to interpret

Lovins stemmer: such an analys can reve featur that ar not eas vis from th vari in th individu gen and can lead to a pictur of expres that is mor biolog transpar and acces to interpres

Paice stemmer: such an analys can rev feat that are not easy vis from the vary in the individ gen and can lead to a pict of express that is mor biolog transp and access to interpret

Porter Summary

- Do stemming and other normalizations help?
  - English: very mixed results. Helps recall but harms precision
    - operative (dentistry) ⇒ oper
    - operational (research) ⇒ oper
    - operating (systems) ⇒ oper
  - Definitely useful for Spanish, German, Finnish, …
    - 30% performance gains for Finnish!
- Full morphological analysis – at most modest benefits for retrieval

>The effect of stemming and normalization depends on the language.

Why precision drops

Different words with **different meanings** may be reduced to the **same stem**.

Example:

- operative *(dentistry)* → `oper`
- operational *(research)* → `oper`
- operating *(systems)* → `oper`

- Stemming is **much more effective for languages with rich morphology, Finnish shows about 30% performance improvement**
- **[**Stemming is more beneficial in languages where **words have many grammatical variations**, because collapsing these variations greatly reduces vocabulary size and improves matching.**]**

Full Morphological Analysis

- Full morphological analysis attempts to:
  - deeply analyze word structure
  - handle all grammatical variations
- However, for IR systems:
  - benefits are **modest**
  - computational cost is high

📌 Therefore, full morphology is often **not worth the extra complexity** for retrieval tasks.

⭐Word (Lexeme) of a Language

- A **word (lexeme)** is the **basic meaningful unit** of a language.
- It represents a **word independent of its grammatical variations.**

> Formation of lexeme is based on:

- Root or Origin
- Antonyms
- Synonyms
- Prefixes / Suffixes uses
- Part of speech
- Function [subject,object,modifier]
- Pronunciation
- Spelling
- Meaning

⭐ Morphology

Morphology is a branch of linguistics that studies:

- the **structure of words**
- how words are **formed**
- how word forms change in a language

English morphology is mainly divided into **two types**:

1. Inflectional Morphology

Definition

Inflectional morphology involves adding **suffixes** to a word **without changing its grammatical category**.

- Meaning remains mostly the same
- Only grammatical information changes

Examples

- Verb tense:
  - walk → walking, walked
- Verb agreement:
  - run → runs
- Noun number:
  - cat → cats

📌 Inflection **does not create a new word**, only a new form.

2. Derivational Morphology

Definition

Derivational morphology involves adding **prefixes or suffixes** that **change the grammatical category or meaning** of a word.

- Creates a **new word**
- May change part of speech

Examples

- nation *(noun)* → national *(adjective)*
- national *(adjective)* → nationalize *(verb)*
- happy *(adjective)* → happiness *(noun)*

📌 Derivation affects **meaning and word class**.

Feature	Inflectional	Derivational
Changes meaning	❌ No	✅ Yes
Changes POS	❌ No	✅ Yes
Creates new word	❌ No	✅ Yes
Example	cats, running	nationalize, happiness

Inflectional vs. Derivational

- Inflectional
  - It is the study of the modification of (lexeme) words to fit into different grammatical contexts
  - It create new form of the same words
  - Use of suffix is common.
  - Semantically regular
- Example
  - Big -> Bigger, Biggest

- Derivational
  - It is the study of the formation of new (lexeme) words that differ either in syntactic category or meaning from their base-words
  - It create new words.
  - Use of prefix /suffix are common
  - Semantically irregular
- Example
  - Activation - >reactivation, Antinational

⭐slide 22 week 2a

Problem (Solution)

- Are the following statements true or false?
  - In a Boolean retrieval system, stemming never lowers precision. (False)
  - In a Boolean retrieval system, stemming never lowers recall. (True)
  - Stemming increases the size of the vocabulary. (False)
  - Stemming should be invoked at indexing time but not while processing a query. (False)

⭐ points discussed in class

If the **result set is very large**, post-processing will take **more time**.
Therefore, **post-processing is practical only when the result set is small**.

1️⃣ What is post-processing?

Post-processing is done **after initial retrieval**, such as:

- ranking
- proximity checking
- phrase checking
- filtering
- snippet generation
- re-scoring

These operations are performed **only on the retrieved documents**.

📌 Time complexity of post-processing is usually:

$$O(|\text{result set}|)$$

So:

- large result set → slow processing
- small result set → fast processing

Exercise 2.3 [⋆] The following pairs of words are stemmed to the same form by the Porter stemmer. Which pairs, would you argue, should not be conflated? Give your reasoning.

a. abandon/abandonment
b. absorbency/absorbent
c. marketing/markets
d. university/universe
e. volume/volumes

a. abandon / abandonment

Should NOT be conflated

Reason:

"Abandon" is a verb.

"Abandonment" is a noun.

They have related meaning but different grammatical and contextual usage.

⇒ **same w (b,c,d)**

e. volume / volumes

Can be conflated

Reason:

Singular and plural forms represent the same concept.

This is inflectional variation.

Exercise 2.4 [⋆] For the Porter stemmer rule group shown in (2.1):

a. What is the purpose of including an identity rule such as SS → SS?

b. Applying just this rule group, what will the following words be stemmed to?

circus *canaries* *boss*

c. What rule should be added to correctly stem *pony*?

d. The stemming for *ponies* and *pony* might seem strange. Does it have a deleterious effect on retrieval? Why or why not?

a. What is the purpose of including an identity rule such as SS → SS?

Purpose:

- Prevent over-stemming.
- Ensure that words ending in "ss" (like "boss") are not incorrectly reduced to "bos".
- Maintains correctness of valid word endings.

---

b. Applying just this rule group, what will the following words be stemmed to?

Using typical plural handling rules:

- circus → `circu`
- canaries → `canari`
- boss → `boss`

Explanation:

- "circus" → final "s" removed
- "canaries" → "ies" → "i"
- "boss" → remains unchanged due to SS → SS rule

c. What rule should be added to correctly stem pony?

Add rule:

```
Code

y → i (when preceded by a vowel)
```

So:

- ponies → poni
- pony → poni

d. The stemming for ponies and pony might seem strange. Does it have a deleterious effect on retrieval? Why or why not?

No, it usually does NOT have a harmful effect.

Reason:

- Retrieval compares stems, not dictionary words.
- Both forms reduce to the same stem (`poni`).
- This improves recall.
- Even though the stem is not a valid English word, consistency is maintained.

⭐ Implementation Issues in IR

- Efficient implementation of an IR system depends heavily on **how the inverted index is stored and accessed**.

› Inverted Index

An **inverted index** maps: `term → list of documents containing the term`

To implement this efficiently, different **data structures** are used

1. Lists

- Posting lists are stored as **ordered lists of document IDs**
- Usually sorted in **increasing order of docID**

Advantages:

- Simple to implement
- Efficient for sequential scanning
- Supports fast intersection (merge-based)

Disadvantages:

- Slow random access
- Inefficient for searching individual elements

📌 Most basic and commonly used structure for postings.

2. HashMap

Used mainly for the **dictionary (term → posting list)**.

- Key = term
- Value = pointer to posting list

Advantages:

- Very fast lookup: **O(1)** average time
- Efficient when vocabulary is large

Disadvantages:

- No ordering of terms
- More memory overhead
- Not suitable for prefix or range queries

📌 Best for fast term lookup.

3. Trees

Commonly implemented using:

- Binary Search Trees (BST)
- B-trees / B+ trees

Advantages:

- Terms are stored in **sorted order**
- Supports:
  - prefix queries
  - range queries
- More memory-efficient than hashmaps in some cases

Disadvantages:

- Lookup is slower than hashmap: **O(log n)**

📌 Often used when ordered access is required.

› Skip Lists:

skip list is an enhancement added to **posting lists** to **speed up merging** and **intersection**.

- Contains **skip pointers** that **allow jumping ahead** in the list
- Reduces unnecessary comparisons

- Only helpful for **AND** queries!!

Example Use

While intersecting two posting lists:

- If current docID is much smaller than the other
- Skip pointer jumps ahead instead of moving one step at a time

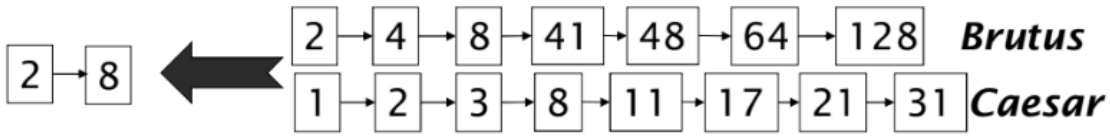
Advantages

- Improves query processing speed
- Especially useful for long posting lists

Disadvantages

- Additional storage overhead
- Slightly more complex implementation

[basic merge:]



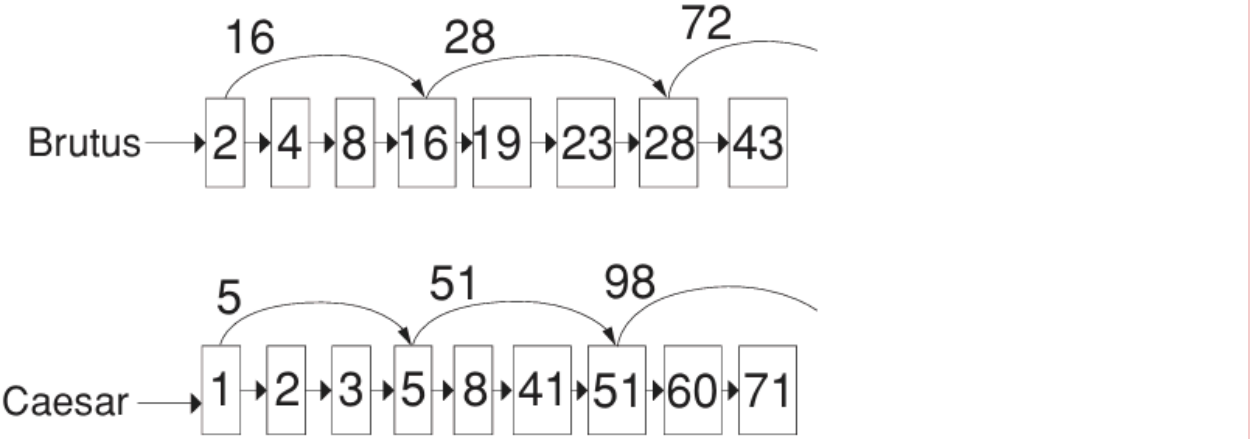
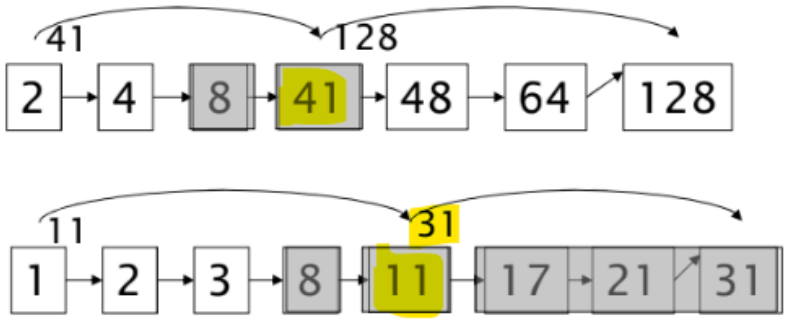


Figure 2.9 Postings lists with skip pointers. The postings intersection can use a skip pointer when the end point is still less than the item on the other list.



Suppose we've stepped through the lists until we process **8** on each list. We match it and advance.

We then have **41** and **11** on the lower. **11** is smaller. But the skip successor of **11** on the lower list is **31**, so we can skip ahead past the intervening postings.

[Algorithm:]

```
IntersectWithSkips(p1, p2)
 1  answer ← ⟨ ⟩
 2  while p1 ≠ NIL and p2 ≠ NIL
 3  do if docID(p1) = docID(p2)
 4        then Add(answer, docID(p1))
 5             p1 ← next(p1)
 6             p2 ← next(p2)
 7        else if docID(p1) < docID(p2)
 8                then if hasSkip(p1) and (docID(skip(p1)) ≤ docID(p2))
 9                        then while hasSkip(p1) and (docID(skip(p1)) ≤ docID(p2))
10                             do p1 ← skip(p1)
11                        else p1 ← next(p1)
12                else if hasSkip(p2) and (docID(skip(p2)) ≤ docID(p1))
13                        then while hasSkip(p2) and (docID(skip(p2)) ≤ docID(p1))
14                             do p2 ← skip(p2)
15                        else p2 ← next(p2)
16  return answer
```

Figure 2.10 Postings lists intersection with skip pointers.

While both lists are not empty:

1. If docIDs match:
   - Add to answer
   - Advance both
2. If docID(p1) < docID(p2):
   - If skip available AND skip docID ≤ docID(p2)
     - Jump using skip
   - Else
     - Move to next
3. Symmetric case for p2

› Where Should We Place Skip Pointers?

- This is a tradeoff problem!

If we place MANY skips:	If we place FEW skips:
✔ Short skip spans	✔ Less memory
✔ More opportunities to skip	✔ Fewer skip comparisons
✘ More memory usage	✘ Longer spans
✘ More comparisons to check skip	✘ Fewer skipping opportunities

› Simple Heuristic (Important!)

For a postings list of length P:

$$\text{Use } \sqrt{P} \text{ evenly spaced skip pointers}$$

Example:

- If P = 10,000
- Use ≈ 100 skip pointers

This balances:

- Memory
- Speed
- Skip efficiency

› Skip pointers work best when: ✔ **Index is static**

› They are harder when:

- Documents are frequently added or deleted
- Posting list changes often

- Skip pointers improve intersection speed by reducing comparisons.
- However, they **increase postings list size.**
- modern systems→**disk I/O** is often the main bottleneck.
- Loading larger posting lists may cost more time than the CPU time saved.
- Therefore, skip pointers are most beneficial in memory-based systems [where index is entirely memory-based]

Exercise 2.5 [⋆] Why are skip pointers not useful for queries of the form x OR y?

Skip pointers help only when we can **skip over entries that cannot possibly match**, which happens in **AND (intersection)**.
For **x OR y (union)**, we must output **every docID that appears in either list**, so we **cannot skip** items — skipping would miss documents that must be included. Hence, skip pointers don't reduce work for OR queries.

Exercise 2.6 [⋆] We have a two-word query. For one term the postings list consists of the following 16 entries:
[4,6,10,12,14,16,18,20,22,32,47,81,120,122,157,180]
and for the other it is the one entry postings list:
[47].
Work out how many comparisons would be done to intersect the two postings lists with the following two strategies. Briefly justify your answers:
a. Using standard postings lists
b. Using postings lists stored with skip pointers, with the suggested skip length of $\sqrt{P}$.

a. Using standard postings lists

Answer: 11 comparisons

Justification (merge intersection):

Compare each element in the long list with 47 until you reach 47:

4, 6, 10, 12, 14, 16, 18, 20, 22, 32, **47**

That's **10 comparisons** to advance up to 32, and the **11th** comparison is 47 vs 47 (match).

b. Using postings lists stored with skip pointers, with the suggested skip length of √P

Answer: 5 comparisons (with √16 = 4 skip length)

Justification:

For P = 16, √P = 4, so we place skips about every 4 postings (typical jumps like):

4 → 14 → 22 → 120

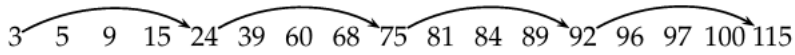
Now intersect with [47]:

- Compare 4 vs 47 → can skip to 14 (≤47)
- Compare 14 vs 47 → can skip to 22 (≤47)
- Compare 22 vs 47 → next skip would go to 120 (>47), so don't skip
- Compare 32 vs 47 → advance
- Compare 47 vs 47 → match

So we only compare: **4, 14, 22, 32, 47** against 47 → **5 comparisons**.

(We skip checking 6,10,12,16,18,20 entirely.)

Exercise 2.7 [⋆] Consider a postings intersection between this postings list, with skip pointers:



and the following intermediate result postings list (which hence has no skip pointers):

3 5 89 95 97 99 100 101

Trace through the postings intersection algorithm in Figure 2.10.

a. How often is a skip pointer followed (i.e., p_1 is advanced to $skip(p_1)$)?

b. How many postings comparisons will be made by this algorithm while intersecting the two lists?

c. How many postings comparisons would be made if the postings lists are intersected without the use of skip pointers?

Given

Postings list p1 (has skip pointers):

`3, 5, 9, 15, 24, 39, 60, 68, 75, 81, 84, 89, 92, 96, 97, 100, 115`

Skips on p1 (from the figure):

- `3 → 24`
- `24 → 75`
- `75 → 92`
- `92 → 115`

Intermediate result postings list p2 (no skips):

`3, 5, 89, 95, 97, 99, 100, 101`

How the intersection proceeds (key steps)

- Start: `p1=3`, `p2=3` → match, advance both
- `p1=5`, `p2=5` → match, advance both
- Now `p2=89`, and p1 moves: `9, 15, 24` (all < 89)

Skip use moment

- At `p1=24` and `p2=89`:
  - check skip: `skip(24)=75`, and **75 ≤ 89**, so **follow skip**
  - jump **24 → 75** (skips 39, 60, 68)
- Now at `p1=75`, `p2=89`:
  - check skip: `skip(75)=92`, but **92 ≤ 89 is false**, so stop skipping
  - then advance normally: 75 → 81 → 84 → 89 (match)

Continue normally to match `97` and `100`.

Final intersection result = **[3, 5, 89, 97, 100]**

(a) How often is a skip pointer followed?

✅ Answer: 1 time → Only one actual skip jump happens: **24 → 75**

(b) How many postings comparisons with the algorithm in Fig 2.10?

This depends on what your teacher counts as a "comparison":

✅ **If you count only comparisons between current docIDs (docID(p1) vs docID(p2)):**

Answer: 16 comparisons

✅ **If you ALSO count skip checks (docID(skip(p1)) ≤ docID(p2)) as comparisons:**

- docID comparisons: 16
- skip-check comparisons: 4

  Total = 20 comparisons

(Those skip checks happen at: 24 vs 89, 75 vs 89, 92 vs 95, etc.)

⇒ **(c)**

#	p1	p2	Result	Move
1	3	3	match	both → (5,5)
2	5	5	match	both → (9,89)
3	9	89	9 < 89	p1 → 15
4	15	89	15 < 89	p1 → 24
5	24	89	24 < 89	p1 → 39
6	39	89	39 < 89	p1 → 60
7	60	89	60 < 89	p1 → 68
8	68	89	68 < 89	p1 → 75
9	75	89	75 < 89	p1 → 81
10	81	89	81 < 89	p1 → 84
11	84	89	84 < 89	p1 → 89
12	89	89	match	both → (92,95)
13	92	95	92 < 95	p1 → 96
14	96	95	96 > 95	p2 → 97
15	96	97	96 < 97	p1 → 97
16	97	97	match	both → (100,99)
17	100	99	100 > 99	p2 → 100
18	100	100	match	both → (115,101)
19	115	101	115 > 101	p2 → end (NIL) stop

✅ Total comparisons = **19**

⭐ Phrase queries

- **phrase query** asks the system to retrieve documents where words appear next to each other in the exact order.

📌**order and adjacency matter**.

- Most recent search engines support a **double quotes syntax** ("stanford university") for phrase queries, but this alone isnt sufficient !!
- Two main approaches:
  1) **bi-word indexing**
  2) **positional indexing**

1️⃣bi-word indexes:

- Index every consecutive pair of terms in the text as a phrase
- Each of these bi-words is now a **dictionary term**
- Fast for two word phrases, but increases index size.
- Can cause false positives for **longer phrases.**

- For example: the text *"Friends, Romans, Countrymen"* would generate the biwords:
  - friends romans
  - romans countrymen

🔟 Problem with Single Terms

Biword index does not naturally handle single-word queries.

To search for:

```
university
```

We would need to scan all biwords containing "university".

Single-word index must still be maintained.

⇒ If using a biword strategy in practice, yes — both a **single-word inverted index** and a **biword index** are typically maintained. The biword index **supports efficient phrase queries**, while the single-word index **supports standard term queries.**

› Longer phrase queries:

- Longer phrases are broken into overlapping biwords
- The query “stanford university palo alto” can be broken into the Boolean query on biwords: “stanford university” and “university palo” and “palo alto”

- **Problem**: **False Positives ⚠️ + index blowup due to bigger dict**
  - Biword matching does not guarantee that the full phrase exists.
  - Even if all component biwords are present:
    - They may not appear contiguously.
    - They may occur in different parts of the document.
  - To verify the full phrase, document inspection is required.
  - Exhaustive phrase indexing (length > 2): Impractical for large collections

1️⃣ 2️⃣ False Positives vs Vocabulary Growth Tradeoff

- Short phrases (length ≥ 3) reduce false positives significantly.
- But storing all possible phrases expands dictionary size heavily.

There is a tradeoff between:

- Accuracy
- Storage cost

- Most search queries involve:
  - Nouns
  - Noun phrases
  - Concept descriptions
- However, related nouns are often separated by function words:

```
abolition of slavery
renegotiation of the constitution
```

^ how do we handle such phrases? **Extended bi-words**

› Extended Biwords (NX*N Model)

1. Perform tokenization.

2. Apply part-of-speech (POS) tagging.
3. Classify words into:
    - **N** → **Nouns** (including proper nouns)
    - **X** → **Function words** (articles, prepositions)

renegotiation	of	the	constitution
N	X	X	N

To process a query:

1. Parse query into N and X categories.
2. Segment into extended biwords.
3. Look up those biwords in the index.

- Parsing may **not always** be **optimal**!

Example:

```
cost overruns on a power plant
```

Parsed into:

```
"cost overruns"
"overruns power"
"power plant"
```

But intuitively, "overruns power" may not be meaningful.

Better POS patterns can improve results.

› Phrase Index:

- Biword index can be extended to longer sequences.

- if **variable-length** word sequences are indexed → called a **phrase index**.

```
machine learning
machine learning is
machine learning is powerful
learning is
learning is powerful
```

Now we are storing **phrases of different lengths**:

- 2-word
- 3-word
- 4-word
- etc.

1️⃣ Why Not Biword Index?

Biword index problems:

- Vocabulary becomes very large
- False positives possible
- Hard to handle long phrases
- Cannot handle proximity queries

So instead, we use:

> **Positional Index (standard solution)**

2️⃣Positional indexes:

- Most commonly used!
- In the postings, store for each term, the position(s) in which **tokens of it appear**
- **for each term** in the index vocabulary, we store **postings** of the form

docID: < position1, position2, … >

<***term***, number of docs containing ***term***;
doc1: position1, position2 … ;
doc2: position1, position2 … ;
etc.>

to, 993427:
⟨ 1, 6: ⟨7, 18, 33, 72, 86, 231⟩;
2, 5: ⟨1, 17, 74, 222, 255⟩;
4, 5: ⟨8, 16, 190, 429, 433⟩;
5, 2: ⟨363, 367⟩;
7, 3: ⟨13, 23, 191⟩; . . . ⟩

be, 178239:
⟨ 1, 2: ⟨17, 25⟩;
4, 5: ⟨17, 191, 291, 430, 434⟩;
5, 3: ⟨14, 19, 101⟩; . . . ⟩

Figure 2.11 Positional index example. The word to has a document frequency 993,477, and occurs six times in document 1 at positions 7, 18, 33, and so on.

Processing a phrase query

- Extract inverted index entries for each distinct term: ***to, be, or, not.***
- Merge their *doc:position* lists to enumerate all positions with "***to be or not to be***".
  - ***to***:
    - *2*:1,17,74,222,551; ***4***:8,16,190,429,433; *7*:13,23,191; ...
  - ***be***:
    - *1*:17,19; ***4***:17,191,291,430,434; *5*:14,19,101; ...
- Same general method for proximity searches

We get postings for:

```
Code

to, be, or, not
```

Then:

- Check adjacency step by step
- Check correct offsets

If:

```
Code

to at pos 10
be at pos 11
or at pos 12
not at pos 13
to at pos 14
be at pos 15
```

- First, find documents that contain ALL terms, we don't check positions yet.
- Now we look at the positions inside doc4.
- suppose doc4 contains all of them.

```
to: 10, 14
be: 11, 15
or: 12
not: 13
```

- Now we check adjacency.

  Take a position of the first word ("to"):

  Example:
  to at position 10

  Now check:

  Is there a "be" at position 11?
  Yes.

  Is there an "or" at position 12?
  Yes.

  Is there a "not" at position 13?
  Yes.

  Is there a "to" at position 14?
  Yes.

  Is there a "be" at position 15?
  Yes.

- So the whole phrase matches starting at position 10.

We are checking:

```
Code

pos(w2) = pos(w1) + 1
pos(w3) = pos(w2) + 1
...
```

We do NOT:

- Scan entire document
- Restart searching for each word randomly

We follow offsets.

[proximity intersection!]

```
POSITIONAL INTERSECT(p1, p2, k)
 1  answer ← ⟨ ⟩
 2  while p1 ≠ NIL and p2 ≠ NIL
 3  do if docID(p1) = docID(p2)
 4        then l ← ⟨ ⟩
 5             pp1 ← positions(p1)
 6             pp2 ← positions(p2)
 7             while pp1 ≠ NIL
 8             do while pp2 ≠ NIL
 9                do if |pos(pp1) − pos(pp2)| ≤ k
10                      then ADD(l, pos(pp2))
11                      else if pos(pp2) > pos(pp1)
12                              then break
13                   pp2 ← next(pp2)
14                while l ≠ ⟨ ⟩ and |l[0] − pos(pp1)| > k
15                do DELETE(l[0])
16                for each ps ∈ l
17                do ADD(answer, ⟨docID(p1), pos(pp1), ps⟩)
18                pp1 ← next(pp1)
19             p1 ← next(p1)
20             p2 ← next(p2)
21        else if docID(p1) < docID(p2)
22                then p1 ← next(p1)
23                else p2 ← next(p2)
24  return answer
```

Figure 2.12 An algorithm for proximity intersection of postings lists p_1 and p_2. The algorithm finds places where the two terms appear within k words of each other and returns a list of triples giving docID and the term position in p_1 and p_2.

- **[2]** Loop while both postings lists still have items
- **[3] If current docIDs match**, then we can check positions
- **[4] Create a temporary list** `l` (empty). This will store candidate positions of term2 that are "close enough" to term1 positions.
- **[5]** Get position list for term1 in this doc:
- **[6]** Get position list for term2 in this doc:
- **[7]** For each position of term1 (`pp1`), we try to find nearby positions of term2.
- **[8]** Scan through positions of term2 (`pp2`) for the current `pp1`.
- **[9] Check proximity condition**: if `|pos(pp1) - pos(pp2)| ≤ k`, then term2 occurs within k words of term1.
- **[10]** If it's **within k**, **add** `pos(pp2)` **to list** `l` (these are "good" term2 positions for the current term1 position).

- [dont get whats happening iske aagey]

› Proximity queries:

- LIMIT! /3 STATUTE /3 FEDERAL /2 TORT
- Again, here, /k means “within k words of”.

/k = at most (k-1) words in between.

Quick examples

`LIMIT! /3 STATUTE`

Allowed patterns (LIMIT and STATUTE can swap order too):

- `LIMIT statute` → 0 words between ✅
- `LIMIT new statute` → 1 word between ✅
- `LIMIT very new statute` → 2 words between ✅
- `LIMIT x y z statute` → 3 words between ❌ (too far)

Because with 3 words in between, distance becomes 4, which violates `/3`.

› Positional index size:

- **Position values can be compressed** (e.g., using gap encoding, variable-byte coding, etc.). [discussed in aagey ke chapters!]
- Storage depends on total number of tokens, not just number of documents -> substantially **increases postings size**.
- Even though positional indexes require more space:
    - They **support exact phrase queries**
        - e.g., "machine learning"
    - They support **proximity queries**
        - e.g., machine /3 learning
    - They **improve ranking quality**
        - Documents where query terms appear close together get higher scores
- Positional indexing is now the **standard approach** in modern search systems!!

- A posting now needs an **entry for each occurrence of a term**. The index size thus depends on the average document size.

In a positional index:

If a word appears 50 times:

We store:

```
term → docID : pos1, pos2, pos3, ..., pos50
```

So storage grows with:

> Total number of word occurrences (tokens)

Not number of documents.

- Index size depends on average document size
  - Average web page has <1000 terms **Why?**
  - SEC filings, books, even some epic poems … easily 100,000 terms
- Consider a term with frequency 0.1%

Document size	Postings	Positional postings
1000	1	1
100,000	1	100

Important Insight

Storage size depends more on:

> Total number of tokens (T)

Than:

> Number of documents (N)

That's why complexity also shifts from O(N) to O(T).

- **2 to 4 times larger** than a simple inverted index.
- A compressed positional index will typically be about **one-third to one-half the size of the original text collection [35–50%]**
- These numbers mainly apply to English-like languages.

⭐ Combination scheme (mixed indexing)

- Combines bi-words/phrase indexes and positional indexes
- In a mixed scheme:
    - We keep a **positional index** for general phrases and proximity search.
    - We also keep **selected frequent phrases** (like common names) as single indexed terms.
- Instead of computing: michael AND jackson (with positional checking)

  We directly look up "michael jackson" as one dictionary entry! [**saves time**]

›4️⃣Research Result (Williams et al., 2004)

- They tested a mixed indexing scheme.
- Now we're maintaining 3 indexes!

1️⃣ Positional index

Standard index storing:

```
Code

term → docID : positions
```

2️⃣ Biword / phrase index

Store common consecutive word pairs:

```
Code

"michael jackson"
"new york"
```

3️⃣ Next-word index (the new idea)

For each word, store which words commonly follow it.

Example:

If the text contains:

```
Code

michael jackson
michael jordan
```

The index stores:

```
Code

michael → jackson, jordan
```

This does NOT store full positions,
just immediate next-word relationships.

- Compared to only using a positional index:
  - Query time reduced to **¼ (25%)**
  - Storage increased by only **26%**

Think of it like:

- Positional index = very flexible but slower
- Biword index = fast for 2-word phrases but large
- Next-word index = smart shortcut for common phrase patterns

⭐<Food for Thoughts>

1. Explain what we mean by document processing pipeline for IR system.

› Docs to be indexed → tokenizer → linguistic modules (stemming,lemmatization) → indexer → inverted index

---

2. What are the two main morphological transformation for English language?

› **inflectional morphology**and **derivational morphology** (e.g., nation → national → nationalize).

---

3. What are the challenges in Document Processing? Explain each with an example.

› Challenges include tokenization (e.g., "Hewlett-Packard"), normalization (e.g., U.S.A. vs USA), handling stop words (e.g., the, is), and multilingual processing (e.g., Chinese without spaces).

---

4. Differentiate between Stemming and Lemmatization of a natural language token(word).

› Stemming removes suffixes using rules (e.g., running → runn), while lemmatization reduces words to dictionary base form using linguistic analysis (e.g., running → run).

---

5. We have a two-word query. For one term the postings list consists of the following 16 entries:

`[4,6,10,12,14,16,18,20,22,32,47,81,120,122,157,180]`

and for the other it is the one entry postings list:

`[47]`

Work out how many comparisons would be done to intersect the two postings lists with the following two strategies. Briefly justify your answers:

a. **Using standard postings lists (no skips)**

› 11 comparisons,

b. **Using postings lists stored with skip pointers (√P ≈ 4)**

4 → 14 (skips over 6,10,12)

14 → 22 (skips over 16,18,20)

22 → 47 (skips over 32)

47 → 157 (skips over 81,120,122)

Comparisons between docIDs (p1 vs 47):

1. 4 vs 47 → try skip (to 14, still ≤47) → jump
2. 14 vs 47 → try skip (to 22, still ≤47) → jump
3. 22 vs 47 → try skip (to 47, still ≤47) → jump
4. 47 vs 47 → **match**, stop

So docID comparisons = **4**.

But we also do skip-check comparisons (checking whether `docID(skip(p1)) ≤ docID(p2)`):

- check skip at 4 (14 ≤ 47) → 1
- check skip at 14 (22 ≤ 47) → 1
- check skip at 22 (47 ≤ 47) → 1

Skip-checks = **3**.

✅ **Total comparisons (docID + skip-checks) = 4 + 3 = 7.**

---

6. What do we mean by extended bi-words? How it is useful?

› Extended bi-words are phrases of the form **NX*N** (**noun–function words–noun**), e.g., "renegotiation of constitution," useful for efficiently handling meaningful noun phrases.

---

7. How positional indexing support phrase and proximity queries? Explain each with an example.

Positional indexing stores term positions, allowing phrase matching (e.g., "stanford university" requires consecutive positions) and proximity queries (e.g., employment /3 law means within 3 words).

---

8. How a combination of bi-word indexing and positional indexing benefits an IR system?

› Combining both allows fast lookup of common phrases (bi-words) while retaining full flexibility for general phrase/proximity search (positional index), improving speed with moderate space increase.

⭐ book exercise:

⇒ **Exercise2.8** : Assume a biword index.Give an example of a document that will be returned for a query of New York University but is actually a false positive that should not be returned

So what would be a false positive in a *biword* index?

A document like:

> "She studied at **New York**. Later, her friend joined **York University** in Canada."

This document contains:

- The biword **New York**
- The biword **York University**

So it will be retrieved.

But it does **not** contain the phrase "New York University" as a single institution. The two bigrams appear in different parts of the document.

✅ Therefore, this is a **false positive under a biword index**.

Exercise 2.9 [⋆] Shown below is a portion of a positional index in the format: term: doc1: ⟨position1, position2, . . . ⟩; doc2: ⟨position1, position2, . . . ⟩; etc.

angels: 2: ⟨36,174,252,651⟩; 4: ⟨12,22,102,432⟩; 7: ⟨17⟩;
fools: 2: ⟨1,17,74,222⟩; 4: ⟨8,78,108,458⟩; 7: ⟨3,13,23,193⟩;
fear: 2: ⟨87,704,722,901⟩; 4: ⟨13,43,113,433⟩; 7: ⟨18,328,528⟩;
in: 2: ⟨3,37,76,444,851⟩; 4: ⟨10,20,110,470,500⟩; 7: ⟨5,15,25,195⟩;
rush: 2: ⟨2,66,194,321,702⟩; 4: ⟨9,69,149,429,569⟩; 7: ⟨4,14,404⟩;
to: 2: ⟨47,86,234,999⟩; 4: ⟨14,24,774,944⟩; 7: ⟨199,319,599,709⟩;
tread: 2: ⟨57,94,333⟩; 4: ⟨15,35,155⟩; 7: ⟨20,320⟩;
where: 2: ⟨67,124,393,1001⟩; 4: ⟨11,41,101,421,431⟩; 7: ⟨16,36,736⟩;

Which document(s) if any match each of the following queries, where each expression within quotes is a phrase query?

a. "fools rush in"
b. "fools rush in" AND "angels fear to tread"

Document 2

- fools: 1, 17, 74, 222
- rush: 2, 66, 194, 321, 702
- in: 3, 37, 76, 444, 851

Check sequence:

- fools 1 → rush 2 → in 3 ✅

So **Doc 2 matches.**

✅ Answer (a): **Documents 2, 4, and 7**

→ similarly compute for doc 4 and 7

Document 4

- angels: 12,22,102,432
- fear: 13,43,113,433
- to: 14,24,774,944
- tread: 15,35,155

Check:

- angels 12
- fear 13
- to 14
- tread 15

Perfect consecutive sequence ✅

So **Doc 4 matches.**

✅ Answer (b): **Document 4 only**

→ doc 2 n 7 dont have this term, so we dont include them in result set as it was an **AND** query!

Exercise 2.10 [⋆] Consider the following fragment of a positional index with the format:

word: document: ⟨position, position, . . .⟩; document: ⟨position, . . .⟩
. . .
Gates: 1: ⟨3⟩; 2: ⟨6⟩; 3: ⟨2,17⟩; 4: ⟨1⟩;
IBM: 4: ⟨3⟩; 7: ⟨14⟩;
Microsoft: 1: ⟨1⟩; 2: ⟨1,21⟩; 3: ⟨3⟩; 5: ⟨16,22,51⟩;

The /k operator, word1 /k word2 finds occurrences of word1 within k words of word2 (on either side), where k is a positive integer argument. Thus $k = 1$ demands that word1 be adjacent to word2.

a. Describe the set of documents that satisfy the query Gates /2 Microsoft.

a) Documents satisfying Gates /2 Microsoft

Check only docs that contain **both** terms: docs **1, 2, 3**.

- **Doc 1**: Gates at 3, Microsoft at 1 → |3−1| = 2 ✅ (within 2)
- **Doc 2**: Gates at 6, Microsoft at 1 or 21 → |6−1|=5, |6−21|=15 ❌
- **Doc 3**: Gates at 2 or 17, Microsoft at 3 → |2−3|=1 ✅ (within 2)

✅ **Answer (a): documents {1, 3}**

b. Describe each set of values for k for which the query Gates /k Microsoft returns a different set of documents as the answer.

Compute the **minimum distance** between any Gates and Microsoft occurrence in each doc:

- **Doc 3**: min |2−3| = **1**
- **Doc 1**: min |3−1| = **2**
- **Doc 2**: min |6−1| = **5** (best possible)

So the returned document set changes when k crosses **1**, **2**, and **5**:

- **k = 1** → docs with min distance ≤ 1 → **{3}**
- **k = 2, 3, 4** → docs with min distance ≤ k → **{1, 3}**
- **k ≥ 5** → includes doc2 as well → **{1, 2, 3}**

✅ **Answer (b):**

- **k = 1** → {3}
- **k ∈ {2,3,4}** → {1,3}
- **k ≥ 5** → {1,2,3}

Exercise 2.11 [⋆⋆] Consider the general procedure for merging two positional postings lists for a given document, to determine the document positions where a document satisfies a $/k$ clause (in general, there can be multiple positions at which each term occurs in a single document). We begin with a pointer to the position of occurrence of each term and move each pointer along the list of occurrences in the document, checking as we do so whether we have a hit for $/k$. Each move of either pointer counts as a step. Let L denote the total number of occurrences of the two terms in the document. What is the big-O complexity of the merge procedure, if we wish to have postings including positions in the result?

You merge the **two position lists inside one document** with two pointers that only move forward. Each move advances one pointer, and each pointer can advance at most the length of its list. Let **L = (#occurrences of term1) + (#occurrences of term2)** in that document.

Even if you also **output positions** (all "hits"), you can generate each hit when you see it while still moving pointers forward; the output size is ≤ O(L) as well.

✅ **Time complexity: O(L)** (linear in the total occurrences in that document).
Space is O(#hits) for the output (or O(1) extra if streaming).

16. Boolean Model Extensions (Recap)

Extension	Key Idea
Extended Boolean Model	Assigns weights (0–1) to terms instead of binary 0/1. Allows partial matching and ranking with soft AND/OR. Combines Boolean logic with vector space concepts.
Fuzzy Set Model	Documents are not just relevant or irrelevant — they have a relevance degree between 0 and 1. Relevance mapped to a membership function.
Proximity Search	Extension via /k operator (within k words). Supported by positional indexing. Used in Westlaw: 'trade secret' /s disclos! /s prevent /s employe!
Ranked Retrieval	Users type free text; system decides best matching documents. No need for Boolean operators. Enables Google-style search. (Full coverage in VSM chapter.)

🔹 Important Concept (Exam Point ⭐)

Normalization must be applied **consistently in both phases**:

Phase	Why Needed?
Document Processing	To store standardized terms
Query Processing	To match user query with stored dictionary terms

If you normalize only documents but NOT queries → lookup fails.